The AI Problem Nobody Warned Senior Developers About

1. Introduction: Why Senior Developers Initially Struggled with AI Agents

When AI coding tools first appeared, many senior developers assumed that years of software engineering experience would give them an immediate advantage.

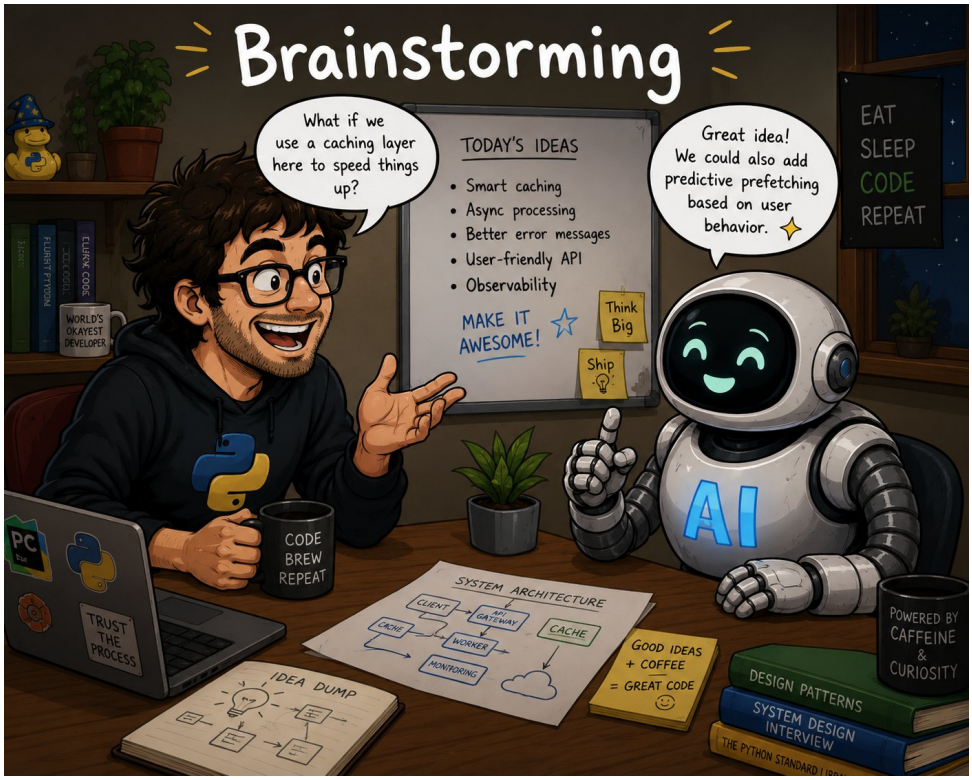
Surprisingly, the opposite often happened and they were disappointed.
Now watch this video to the end to learn our secrets.

Developers with decades of experience found themselves fighting the AI, generating inconsistent results, wasting tokens, and spending hours correcting code that should have been right the first time.

The problem nobody warned senior developers about is that AI does not reward experience alone.

AI rewards clarity, specifications, constraints, and orchestration.

1.1 The concept of Brainstorming



Imagine you wanted a builder to construct your house, and all you told the builder was, "Build me a house."

The problem is that they don't know whether you want the house on stilts, whether it should be a single-story or double-story house, or even what materials should be used.

What you need to do is discuss your requirements with the builder.

Today, we have the equivalent of that process for AI agents, and we call it **brainstorming**.

This is a far better approach than simply typing large prompts and hoping for the best.

In this video, I will show you the workflow that helped me achieve significantly better results with AI agents by focusing on requirements, task planning, skills, and orchestration rather than prompt engineering alone.

2. The Improved Brainstorm

The way I tackle this is by creating a file called `requirements.md`.

In this file, I briefly describe what I want to build.

I also include the technology stack. This includes the programming language I am using, such as Python or Java, along with all the technologies, frameworks, and versions involved in the project.

I then ask the AI agent to discuss the requirements with me.

At this stage, the agent will ask many questions about the project, such as:

* What am I doing on the backend?

* What am I doing on the frontend?

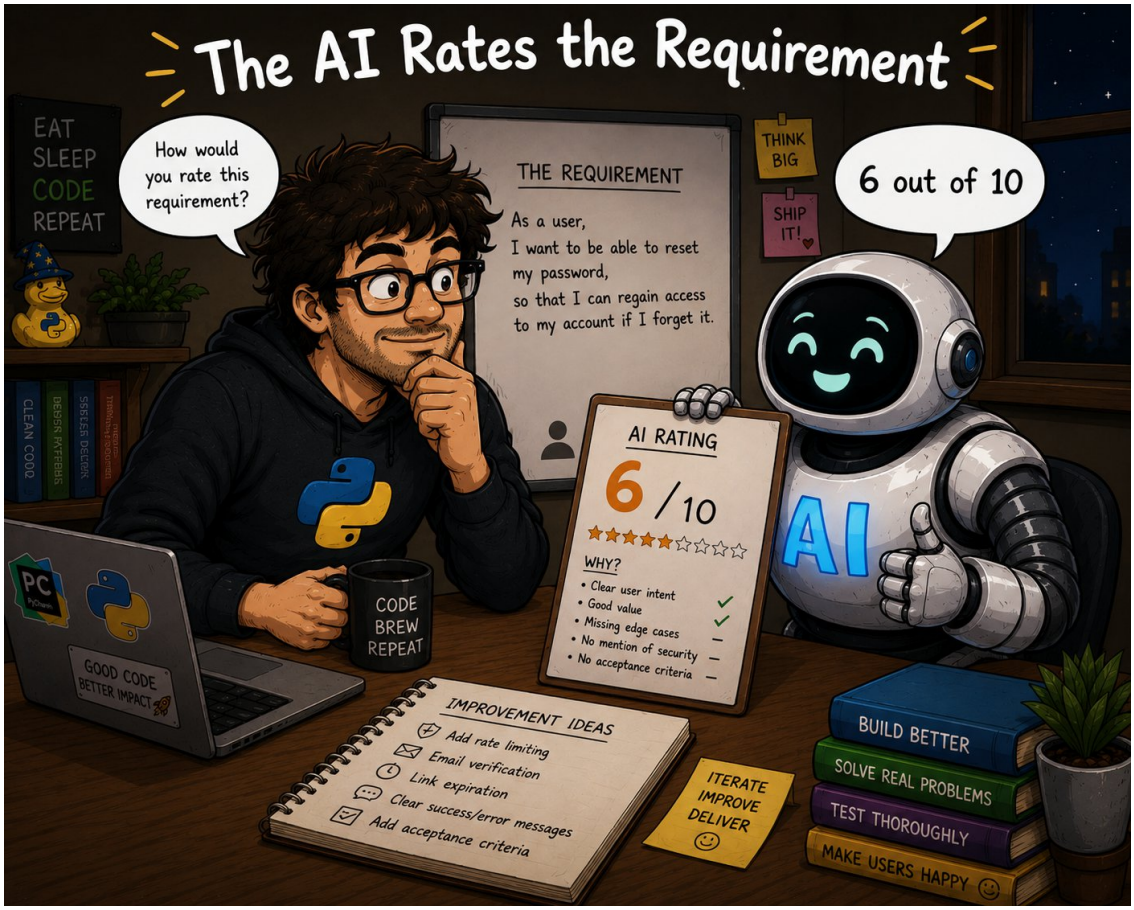
* What database am I using?

* What are the functional and non-functional requirements?

The goal is to build a complete picture that will eventually become the project specification.

Once the discussion is complete, I ask the agent to update my `requirements.md` file so that the conversation and decisions are recorded.

2.1 Get the AI to Rate the Requirements



Next, I ask the agent to evaluate the requirements document and score it out of 10.

Initially, I typically receive a score between 6 and 7 out of 10.

However, the agent will also explain exactly what is missing and what improvements are needed to achieve a 10 out of 10 score.

I then review those recommendations, decide which ones are relevant to my project, and tell the agent what I want to include or exclude.

After updating the document, I ask the agent to evaluate it again.

At this point, I usually receive a score between 8 and 9.5 out of 10, which indicates that the requirements are clear and comprehensive.

3. Creating the Tasks with Their Phases

At this stage, I ask the agent to create a file called `tasks.md`.

In this file, the agent generates all the tasks required to implement the requirements.

I also ask it to group the work into phases.

This provides a structured road-map for the project and allows progress to be tracked more effectively.

At this point, we are ready to build the application.

I can now use a single AI agent to build the project, working through one phase at a time.

4. Using a Single Agent or Multiple Agents to Build the Project

Alternatively, I can use multiple agents working in parallel.

For example, imagine I am building a Python application with an Angular frontend and a database.

I might assign:

* One agent to develop the Python backend.

* One agent to test the Python backend.

* One agent to develop the Angular frontend.

* One agent to test the Angular frontend.

* One agent to design and build the database.

* One agent to create the RESTful APIs and Swagger documentation.

* One agent to test the APIs using Swagger and automated API tests.

Finally, I will have an **orchestrator agent**.

The orchestrator communicates with all the other agents. It coordinates the frontend developers, backend developers, database developers, API developers, and testing agents.

The orchestrator ensures that all teams remain aligned and that the project moves forward in a coordinated manner.

In effect, the orchestrator runs the entire show.

Using this approach, I typically achieve an 80% to 90% success rate.

These AI agents are extremely capable. However, they behave much like very enthusiastic junior developers. They require guidance, supervision, and clear boundaries.

Without sufficient control, they may generate code that introduces technical debt or causes problems elsewhere in the project.

5. Using Skills and Policy Files to Constrain How the Agents Build

To maintain control, we create **skills** and **policy files**.

Skills teach AI agents how to write code according to our engineering standards, architectural patterns, and best practices.

When using multiple agents, each agent should have skills tailored to its role.

For example:

* Development agents should have coding standards and architecture skills.

* Testing agents should have testing and quality assurance skills.

* Database agents should have database design skills.

* API agents should have API design and documentation skills.

This provides tight control over what is built and how it is built.

6. The Developer's Experience Determines the Quality of the Outcome

One observation is that junior developers working with AI often focus heavily on generating code quickly.

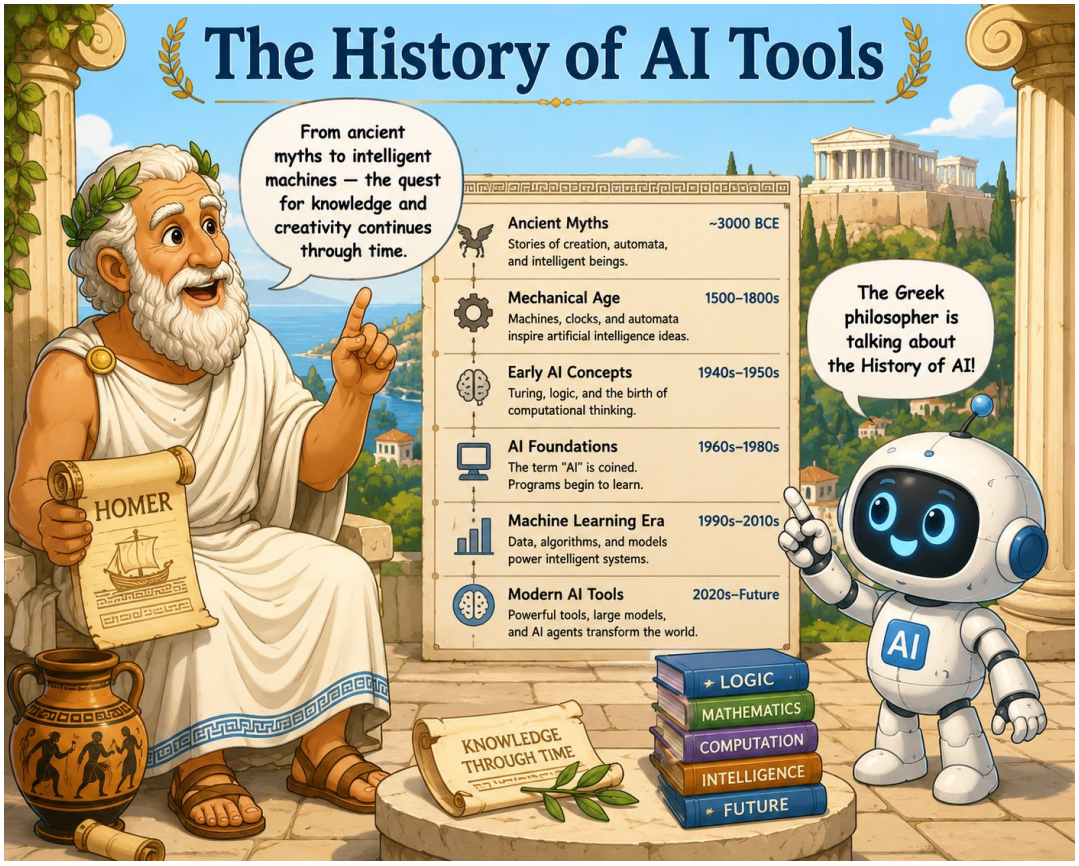
They do not always focus on the industry best practices required to build maintainable software.

As more experienced developers, we are concerned about code quality, maintainability, security, testing, scalability, and long-term support.

These considerations become even more important when working with AI agents.

Good AI-assisted development still requires good software engineering practices.

7. The Tools Used from a Historical Perspective



7.1 GitHub Copilot (Prompt-Driven Development)

Many of us started with GitHub Copilot and early forms of specification-driven development.

The common approach was to create large prompt files and extensive instruction sets.

While this worked, the **prompts quickly became difficult to maintain** as **projects grew in size and complexity.**

7.2 Spec Kit

Tools such as Spec Kit improved the process by encouraging developers to define requirements before generating code.

This was a major step forward because it moved developers away from prompt-driven development and toward specification-driven development.

However, these tools were not designed around multi-agent workflows.

7.3 Multi-Agent Development

The next generation of tools, including

1. Claude Code
2. Cursor
3. Codex
   introduced multi-agent capabilities.

These tools enabled **requirements-driven** and **context-driven** development using files such as

- **`requirements.md`**
- **`tasks.md`.**

By this stage, we had largely solved the productivity problem.

By this stage, we had largely solved the productivity problem.

We had moved beyond prompt engineering and adopted requirements-driven, context-driven, and multi-agent development workflows.

However, a new challenge emerged.

The more successful we became with AI-assisted development, the more tokens we consumed.

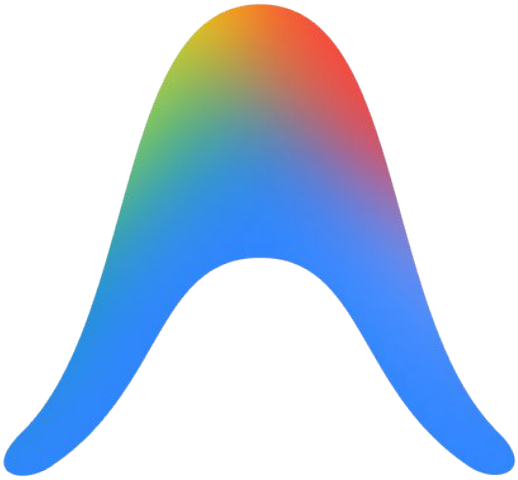
Large specifications, multiple agents, testing agents, orchestrators, and extended project memory all increased costs.

7.3.1 Incurring Expensive Token costs

The productivity gains were significant, but they introduced a new challenge: increased token consumption and higher operating costs.

This led many teams to search for ways to maintain productivity while reducing the cost of AI-assisted development.

8. Anti Gravity: Breaking Free from Token Cost Gravity



Now Google takes a swipe at the Expensive AI Agentic Tools

The productivity gains were real, but so were the expenses.

This is where Anti Gravity enters the picture.

Google has introduced its "Anti Gravity" initiative with the goal of reducing token costs while maintaining the benefits of modern AI-assisted software development.

If successful, this could make large-scale AI development significantly more affordable while preserving the productivity improvements developers have come to expect.

For organizations adopting multi-agent workflows, lowering token costs may become just as important as improving model capabilities.

9. Conclusion

The secret to successful AI-assisted software development is not better prompting—it is better specifications, structured requirements, phased task planning, and disciplined orchestration.

The problem nobody warned senior developers about is that AI does not automatically reward experience.

It rewards clarity.

The more effectively you can define requirements, create constraints, organize tasks, and coordinate agents, the more successful your AI-assisted projects will be.

AI agents are powerful tools, but like any development team, they perform best when given clear requirements, proper supervision, and well-defined standards.

10. Installing and Using Anti Gravity

And now for now the bones chapter